

Vue 3 中 onLoad 与 onReady 生命周期解析

需要明确的是，Vue 3 原生组件体系中并不存在 onLoad 和 onReady 这两个生命周期钩子——它们本质是小程序（如微信小程序）及跨端框架（如 uni-app）衍生的钩子。在 Vue 3 开发中，这两个钩子的含义、用法完全依赖具体场景（纯网页开发 / 跨端开发），以下分场景详细说明核心逻辑与实践方式。

一、纯 Vue 3 网页开发（无跨端需求）

Vue 3 原生生命周期以组件挂载、更新、卸载为核心，onLoad/onReady 的功能需通过原生钩子组合实现。核心对应关系如下：

场景化需求（类似 onLoad/onReady）	Vue 3 原生实现方案	核心作用
页面/组件初始化（接收参数、请求数据）	onMounted	组件挂载到 DOM 树后执行，此时可访问组件内部 DOM 元素
确保 DOM 完全渲染（操作 DOM 元素、初始化插件）	onMounted + nextTick	等待当前渲染周期的 DOM 更新完成，避免操作未渲染的元素

1. 组合式 API（<script setup> 推荐用法）

组合式 API 需从 vue 中引入相关钩子，通过 ref 关联 DOM 元素，结合 nextTick 确保 DOM 就绪：

Code block

```
1  <template>
2    <div ref="contentRef" class="container">Vue 3 原生组件</div>
3  </template>
4
5  <script setup>
6    import { ref, onMounted, nextTick } from 'vue'
7    import { useRoute } from 'vue-router' // 路由参数接收
8
9    // 1. 路由参数接收（类似 onLoad 接收参数）
10   const route = useRoute()
11   const goodsId = route.query.id // 从路由获取参数
12
13   // 2. DOM 元素关联
```

```

14  const contentRef = ref(null)
15
16  // 3. 组件挂载后执行（核心逻辑入口）
17  onMounted(async () => {
18    console.log('组件已挂载，可执行初始化逻辑（对应 onLoad）')
19    console.log('获取的路由参数：', goodsId)
20
21    // 4. 等待 DOM 完全渲染（对应 onReady 场景）
22    await nextTick()
23    console.log('DOM 完全就绪，可操作元素：', contentRef.value.offsetHeight)
24
25    // 实际业务：请求数据、初始化第三方插件等
26    // const res = await fetchData(goodsId)
27  })
28  </script>

```

2. 选项式 API（兼容 Vue 2 写法）

Code block

```

1  <template>
2    <div ref="contentRef" class="container">Vue 3 选项式 API</div>
3  </template>
4
5  <script>
6  export default {
7    // 组件挂载后执行
8    mounted() {
9      console.log('组件挂载完成（对应 onLoad）')
10     console.log('路由参数：', this.$route.query.id)
11
12     // 等待 DOM 完全渲染
13     this.$nextTick(() => {
14       console.log('DOM 就绪（对应 onReady）：', this.$refs.contentRef)
15     })
16   }
17 }
18 </script>

```

二、uni-app + Vue 3 跨端开发（小程序/APP/H5）

uni-app 为适配小程序开发习惯，在 Vue 3 中保留了 onLoad/onReady 钩子，且对页面与组件做了明确区分——这两个钩子仅作用于页面组件，普通组件仍使用 Vue 原生生命周期。

1. 核心钩子说明

钩子名称	触发时机	触发次数	核心使用场景
onLoad	页面加载时（路由跳转后立即触发）	仅 1 次	接收页面参数、初调用接口请求
onReady	页面初次渲染完成后	仅 1 次	操作页面 DOM、初始化图表等第三方插件

2. 组合式 API 实践（<script setup>）

uni-app 需从 @dcloudio/uni-app 引入页面生命周期钩子，支持与 Vue 原生钩子混用：

Code block

```
1  <template>
2    <view ref="pageRef" class="page-container">
3      商品详情页（ID: {{ goodsId }}）
4    </view>
5  </template>
6
7  <script setup>
8    import { ref } from 'vue'
9    import { onLoad, onReady, onShow } from '@dcloudio/uni-app'
10   import { getGoodsDetail } from '@/api'
11
12   const goodsId = ref('')
13   const pageRef = ref(null)
14
15   // 1. 页面加载：接收参数并初始化数据（核心入口）
16   onLoad((options) => {
17     // options 为页面跳转参数（如：从列表页传递 id=123）
18     goodsId.value = options.id
19     console.log('页面加载 onLoad, 商品ID: ', goodsId.value)
20
21     // 立即请求商品详情
22     initGoodsData()
23   })
24
25   // 2. 页面渲染完成：操作 DOM 或初始化插件
26   onReady(() => {
27     console.log('页面渲染完成 onReady, DOM 尺寸: ', pageRef.value)
28     // 示例：初始化地图插件
29     // initMap()
30   })
31
32   // 3. 封装数据请求逻辑
33   const initGoodsData = async () => {
```

```
34     try {
35         const res = await getGoodsDetail(goodsId.value)
36         // 处理接口返回数据
37     } catch (err) {
38         uni.showToast({ title: '请求失败', icon: 'none' })
39     }
40 }
41 </script>
```

3. 选项式 API 实践

Code block

```
1  <template>
2    <view class="page-container">uni-app 选项式 API 页面</view>
3  </template>
4
5  <script>
6  export default {
7    data() {
8      return {
9        goodsId: ''
10      }
11    },
12    // 页面生命周期
13    onLoad(options) {
14      this.goodsId = options.id
15      this.initGoodsData()
16    },
17    onReady() {
18      console.log('页面渲染完成 onReady')
19    },
20    // Vue 原生生命周期（可混用）
21    mounted() {
22      console.log('Vue 原生 mounted 触发')
23    },
24    methods: {
25      initGoodsData() {
26        // 接口请求逻辑
27      }
28    }
29  }
30  </script>
```

三、关键差异与避坑指南

1. 执行顺序（uni-app + Vue 3 场景）

页面加载时钩子执行顺序固定，需根据需求选择对应钩子：

onLoad（页面加载）→ Vue mounted（组件挂载）→ onReady（页面渲染完成）

因此：数据初始化优先用 onLoad，DOM 操作优先用 onReady（比 mounted 更晚，确保 DOM 完全就绪）。

2. 路由参数接收差异

- **纯 Vue 3 网页**：通过 Vue Router 的 useRoute()（组合式）或 this.\$route（选项式）获取参数；
- **uni-app 跨端**：直接通过 onLoad 的 options 参数获取，无需引入路由工具，更贴合小程序开发习惯。

3. 组件与页面的钩子区分



重要提醒：uni-app 中，onLoad/onReady 仅作用于 **页面组件**（pages 目录下的文件）；普通组件（components 目录下）无法使用这两个钩子，需改用 Vue 原生的 onMounted、onUpdated 等。

四、总结：核心场景适配表

开发场景	初始化数据/接收参数	操作 DOM/初始化插件
纯 Vue 3 网页（组合式）	onMounted + useRoute()	onMounted + nextTick()
纯 Vue 3 网页（选项式）	mounted + this.\$route	mounted + this.\$nextTick()
uni-app + Vue 3（页面组件）	onLoad(options)	onReady()
uni-app + Vue 3（普通组件）	onMounted	onMounted + nextTick()

（注：文档部分内容可能由 AI 生成）